Estratégia Concursos

Aula 14

DataPrev (Analista de TI - Desenvolvimento de Software) Passo Estratégico de Conhecimentos Específicos - 2024 (Pós-Edital)

Autor:

09 de Setembro de 2024

11225354498 - Maria Vitória Gonçalves Vieira

JAVA EE

Sumário

Conteúdo 2
Glossário de termos 3
Roteiro de revisão 5
- Introdução 5
- Servlets 9
- JSP 11
- JSF 14
- JPA 20
- EJB 28
- JTA 35
- JMS 41
- JNDI 46

Aposta estratégica 48
Questões Estratégicas 48
Questionário de revisão e aperfeiçoamento 56
- Perguntas 56
- Perguntas e Respostas 57

Lista de Questões Estratégicas 59
- Gabaritos 63

CONTEÚDO

Java EE. Servlets. JSP. Especificações JSF, JPA, JTA, EJB, JMS e JNDI.

ANÁLISE ESTATÍSTICA

Inicialmente, convém destacar o percentual de incidência do assunto, dentro da disciplina **Desenvolvimento de Sistemas e Programação** em concursos/cargos similares. Quanto maior o percentual de cobrança de um dado assunto, maior sua importância.

Obs.: *um mesmo assunto pode ser classificado em mais de um tópico devido à multidisciplinaridade de conteúdo.*

11225354498 - Maria Vitória Gonçalves Vieira

Assunto	Relevância na disciplina em concursos similares
Linguagens de programação	**35.7 %**
Python	**14.5 %**
JavaScript	**12.7 %**
Java	**3.6 %**
PHP	**2.5%**
VB Script	**1.8 %**
C Sharp	**1.8 %**
R	**1.8 %**
Web	**5.5 %**
Linguagens de marcação	**5.5 %**
XML (Extensible Markup Language)	**3.6 %**
HTML (HyperText Markup Language)	**1.8 %**
CSS (Cascading Style Sheets)	**3.6 %**
Conceitos básicos de programação	**1.8 %**
Frameworks Java	**1.8 %**
Hibernate	**1.8 %**

Glossário de termos

Faremos uma lista de termos que são relevantes ao entendimento do assunto desta aula. Caso tenha alguma dúvida durante a leitura, esta seção pode lhe ajudar a esclarecer.

11225354498 - Maria Vitória Gonçalves Vieira

J2EE: Java 2 Platform, Enterprise Edition - É uma plataforma anteriormente usada para desenvolvimento e execução de aplicações empresariais distribuídas. Foi sucedida por Java EE.

Java EE: Java Platform, Enterprise Edition - É a plataforma atual para o desenvolvimento de aplicações empresariais robustas e escaláveis. Suporta várias especificações como Servlets, EJB, JPA, entre outros.

Jakarta EE: A evolução do Java EE sob a égide da Eclipse Foundation. Visa modernizar e desenvolver as tecnologias empresariais baseadas em Java.

EFSP: Eclipse Foundation Specification Process - Processo utilizado pela Eclipse Foundation para o desenvolvimento de especificações, incluindo Jakarta EE.

Servlet: Componentes do lado servidor que lidam com requisições HTTP. São extensões dos servidores que melhoram suas funcionalidades.

JSP: JavaServer Pages - Tecnologia que permite a incorporação de código Java em páginas HTML.

Scriplet: Trechos de código Java incrustados em um documento JSP.

JSF: JavaServer Faces - Framework para a construção de interfaces de usuário em aplicações web.

Bean: Um objeto que segue certas convenções, como ter um construtor sem argumentos e fornecer métodos getter e setter.

JPA: Java Persistence API - Interface de persistência em Java para gerenciar dados relacionais.

JPQL: Java Persistence Query Language - Linguagem de consulta para fazer queries em entidades gerenciadas pela JPA.

JPCA: Java Persistence Criteria API - API para criar queries dinâmicas usando JPA.

Entity Manager: Componente central na JPA para gerenciar entidades.

EJB: Enterprise JavaBeans - Componentes do lado servidor que encapsulam a lógica de negócios de uma aplicação.

11225354498 - Maria Vitória Gonçalves Vieira

IoC: Inversão de Controle - Padrão de design em que o controle do fluxo do programa é transferido para um container ou framework.

JTA: Java Transaction API - API para gerenciamento de transações distribuídas em aplicações Java EE.

2PC: Two-Phase Commit - Protocolo para garantir transações distribuídas atomicamente.

JMS: Java Message Service - API para troca de mensagens entre aplicações distribuídas.

P2P: Point-to-Point - Modelo de comunicação onde cada mensagem é enviada de um ponto (produtor) para outro ponto (consumidor).

Pub-Sub: Publish-Subscribe - Modelo de comunicação onde mensagens são publicadas em tópicos para múltiplos assinantes.

JNDI: Java Naming and Directory Interface - API para descoberta e busca de serviços e objetos em uma rede.

ROTEIRO DE REVISÃO

A ideia desta seção é apresentar um roteiro para que você realize uma revisão completa do assunto e, ao mesmo tempo, destacar aspectos do conteúdo que merecem atenção.

Introdução

Java EE, ou Java Platform, Enterprise Edition, é um conjunto de especificações que estendem a plataforma Java SE (Standard Edition) com especificações para recursos corporativos. Ele oferece uma arquitetura robusta e escalável para a implementação de aplicações corporativas (enterprise) altamente disponíveis, distribuídas e transacionais.

O Java EE engloba uma gama ampla de tecnologias e APIs destinadas a suportar o desenvolvimento de aplicações profissionais. Ele é voltado principalmente para

11225354498 - Maria Vitória Gonçalves Vieira

sistemas que necessitam de escalabilidade, confiabilidade e disponibilidade, comumente exigidas nestes ambientes.

Além disso, Java EE tornou-se um padrão de fato para a construção de grandes aplicações que sejam robustas e escaláveis. É mantido e desenvolvido sob a égide da Eclipse Foundation, através do projeto Jakarta EE, após sua transição da Oracle.

Diferenciais:

- Portabilidade: Podendo ser executado em qualquer servidor que cumpra as especificações Java EE.
- Escalabilidade: Capaz de lidar com demandas crescentes, seja em volume de dados ou em número de usuários.
- Segurança: Oferece uma série de recursos para autenticação, autorização e comunicação segura.
- Integração: Facilita a integração com outros sistemas, sejam eles baseados em Java ou não.

Ao longo da nossa aula veremos em maiores detalhes as seguintes especificações:

- Java Persistence API (JPA): Utilizado para mapeamento objeto-relacional, proporcionando uma forma de persistir objetos Java em um banco de dados relacional.
- Enterprise JavaBeans (EJB): Fornece um modelo de componentes para desenvolvimento de aplicações distribuídas e transacionais.
- JavaServer Faces (JSF): É uma tecnologia para desenvolvimento de interfaces de usuário em aplicações web.
- Java Message Service (JMS): É uma API que permite o envio e o recebimento de mensagens de forma assíncrona entre componentes distribuídos.
- Java Transaction API (JTA): Permite o gerenciamento de transações de forma declarativa, assegurando que as operações sejam completadas ou revertidas de forma adequada.

Histórico: J2EE, Java EE e Jakarta EE

Java EE passou por várias mudanças de nome, incluindo Java 2EE e J2EE, e agora é conhecido como Jakarta EE. A primeira versão do Java incluía extensões corporativas como parte do JDK principal. No entanto, em 1999, essas extensões foram separadas e deram origem ao J2EE.

11225354498 - Maria Vitória Gonçalves Vieira

Em 2006, o J2EE foi renomeado para Java EE, um nome que foi mantido até setembro de 2017. Nesse período, a Oracle transferiu os direitos do Java EE para a Eclipse Foundation, embora ainda mantenha a propriedade sobre a linguagem Java em si.

Devido aos direitos da Oracle sobre a marca "Java", a Eclipse Foundation teve que renomear o Java EE para Jakarta EE. Ainda há incertezas e discussões em curso sobre questões como documentação e direitos autorais.

O processo de adição de novas especificações também mudou. Anteriormente, era regido pelo Java Community Process (JCP) e agora é conduzido pelo Eclipse Foundation Specification Process (EFSP). Este último processo é mais aberto e centrado no código, em contraste com o processo anterior.

Historicamente, uma nova funcionalidade no EE exigia três coisas: uma especificação, uma implementação de referência e testes. Esses elementos ainda são relevantes, mas o EFSP trouxe mais transparência e neutralidade.

O TCK (Technology Compatibility Kit) ainda é utilizado para validar as novas funcionalidades, e tanto este quanto os JSRs (Java Specification Requests) foram tornados open-source pela Oracle.

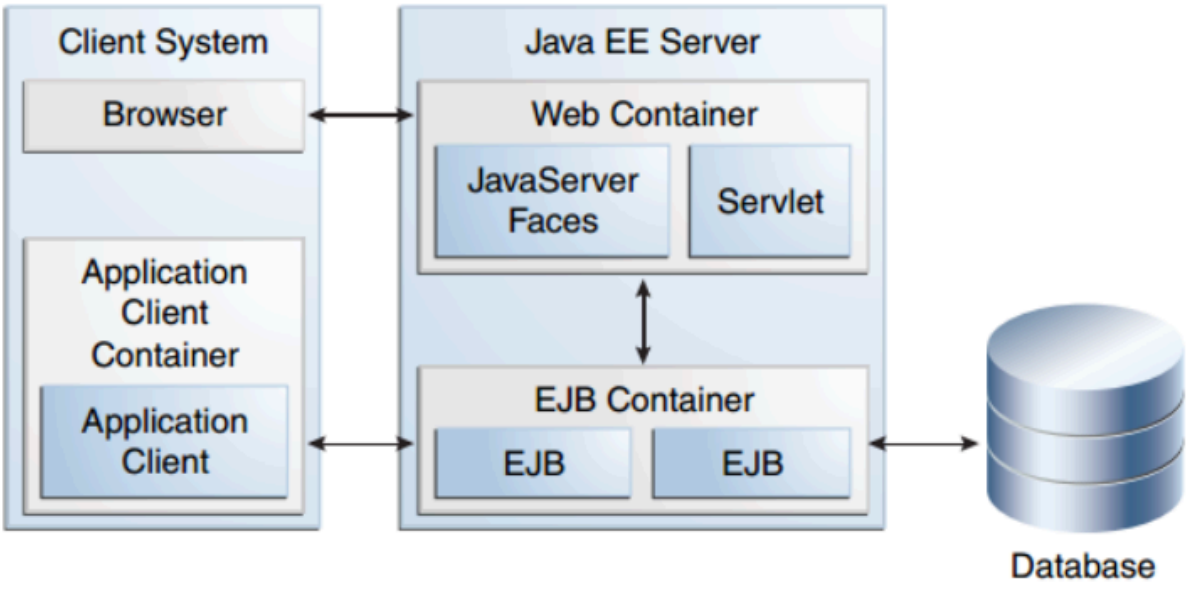
A tabela abaixo mostra um resumo das evoluções de Java EE:

Version	Date
J2EE 1.2	December 1999
J2EE 1.3	September 2001
J2EE 1.4	November 2003
Java EE 5	May 2006
Java EE 6	December 2009
Java EE 7	April 2013
Java EE 8	August 2017
Jakarta EE	February 2018*

Arquitetura Java EE

11225354498 - Maria Vitória Gonçalves Vieira

A arquitetura do Java EE é dividida em diversas camadas e contêineres para separar as responsabilidades e promover uma melhor organização do código e dos recursos. Vamos explorar as principais partes.



Camada Cliente (Client System)

- Aplicativo Web Cliente: Pode ser um navegador web rodando aplicações baseadas em HTML, JavaScript e outras tecnologias de front-end.
- Aplicativo Cliente Java: São programas que rodam em uma máquina cliente e acessam os recursos do servidor Java EE.
- Outros tipos de Clientes: Dispositivos móveis, aplicações de desktop, outros servidores, etc.

Web Tier (Dentro do Java EE Server)

- Web Container: É o componente que gerencia a execução das Servlets e das páginas JSP. O container web também é responsável pelo ciclo de vida, segurança e roteamento das requisições.
- Tecnologias Comuns: Servlets, JSP (JavaServer Pages), JSF (JavaServer Faces), RESTful Web Services (JAX-RS), etc.
- Comunicação: Normalmente, comunica-se com a camada de negócios através de EJBs ou diretamente com a camada de persistência via JPA.

Business Tier (Dentro do Java EE Server)

11225354498 - Maria Vitória Gonçalves Vieira

- **EJB Container**: Gerencia os Enterprise JavaBeans (EJBs), os quais encapsulam a lógica de negócio da aplicação.
- **Tecnologias Comuns**: Session Beans, Message-Driven Beans, Entity Beans (obsoletos em favor do JPA)
- **Transações**: Controladas através da Java Transaction API (JTA)
- **Segurança**: Autenticação e autorização de usuários.

EIS Tier (Enterprise Information System Tier)

- **Recursos de Persistência**: Como bancos de dados, sistemas de mensageria, ERPs, etc.
- **Acesso a Dados**: Via JPA, JMS, JCA (Java Connector Architecture) e outras APIs de integração.

Fluxo Comum de Execução

1. **Requisição do Cliente**: Um cliente faz uma requisição para o aplicativo.
2. **Processamento Web**: O Web Container recebe a requisição e, dependendo do mapeamento, encaminha para uma Servlet, JSP ou recurso JSF correspondente.
3. **Lógica de Negócios**: O EJB Container gerencia os EJBs que executam a lógica de negócios, transações e acesso a dados.
4. **Acesso aos Dados**: Se necessário, os EJBs interagem com a camada de persistência, geralmente usando JPA ou outros meios de acesso a dados.
5. **Resposta ao Cliente**: O resultado é então enviado de volta ao cliente através do Web Container.

Servlets

Servlets são componentes de servidor em Java que estendem as capacidades de servidores que hospedam aplicações baseadas na web. Eles são usados para gerar conteúdo dinâmico, como HTML, XML ou JSON, em resposta a requisições de clientes, que normalmente são navegadores da web. Um servlet é essencialmente uma classe Java que processa uma solicitação HTTP e gera uma resposta HTTP.

Funcionamento Básico

Quando uma solicitação HTTP chega a um servidor web que suporta servlets, o servidor invoca o método apropriado (**doGet**, **doPost**, **doPut**, etc.) no servlet. O servlet processa a solicitação, gera uma resposta e a envia de volta ao cliente através do

11225354498 - Maria Vitória Gonçalves Vieira

servidor web. Toda essa operação é facilitada pelo contêiner de servlets, uma parte essencial de servidores web como Apache Tomcat e Glassfish.

Veja um exemplo de Servlet:

```
import java.io.*;
import java.text.*;
import java.util.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class ServletTeste extends HttpServlet
{
    public void doGet(HttpServletRequest req, HttpServletResponse res)
        throws IOException, ServletException {

            res.setContentType("text/html");
            PrintWriter out = res.getWriter();

            out.println("<html>");
            out.println("<body bgcolor = \"white\">");
            out.println("<h1>Hello Servlet</h1>");
            out.println("</body>");
            out.println("</html>");
    }
}
```

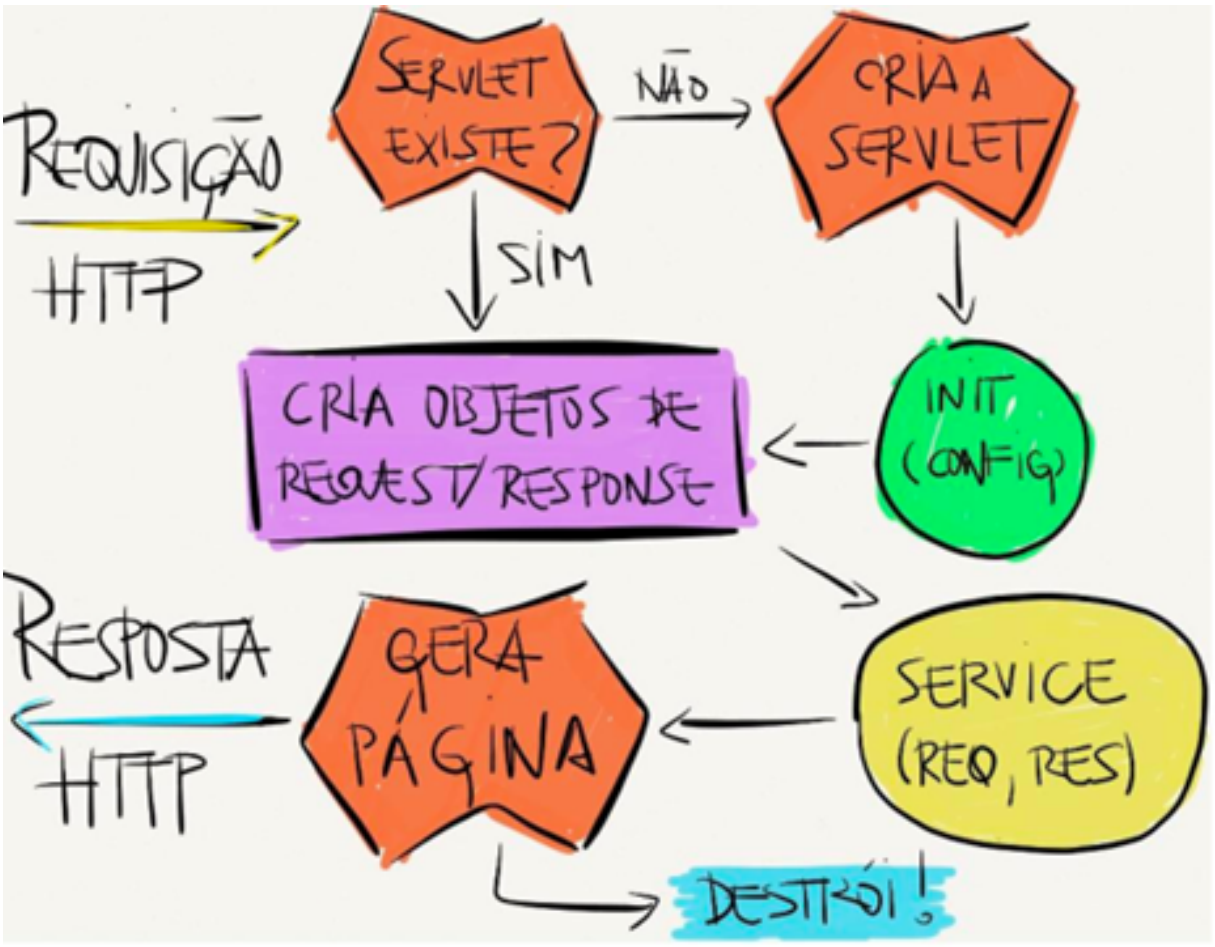
Ciclo de Vida

O ciclo de vida de um servlet é gerenciado pelo contêiner de servlets e geralmente envolve as seguintes fases:

1. Inicialização: Quando o servlet é carregado pela primeira vez, o contêiner invoca o método init().
2. Manipulação de Requisições: Para cada requisição recebida, o contêiner invoca um dos métodos de serviço (doGet, doPost, etc.).
3. Destruição: Quando o servlet é removido da memória, o método destroy() é chamado.

Veja o fluxo de uma requisição utilizando Servlets:

11225354498 - Maria Vitória Gonçalves Vieira



Vantagens e Uso

Servlets oferecem diversas vantagens, tais como eficiência, portabilidade entre diferentes servidores web e segurança. Além disso, como são escritos em Java, eles têm acesso a todo o conjunto de APIs Java.

Servlets são geralmente usados em combinação com tecnologias como JSP (JavaServer Pages) para criar aplicações web. Também podem ser uma parte integrante de frameworks mais complexos como o Spring.

Possíveis problemas

Embora servlets sejam flexíveis, eles também têm suas limitações. Por exemplo, escrever servlets pode se tornar trabalhoso para tarefas muito complexas. Além disso, eles são uma tecnologia do lado do servidor, o que significa que qualquer mudança requer um ciclo completo de solicitação-resposta para ser refletida no cliente.

11225354498 - Maria Vitória Gonçalves Vieira

JSP

JavaServer Pages (JSP) é uma tecnologia usada para desenvolver aplicações web dinâmicas e interativas em Java. JSP permite que desenvolvedores embutam código Java diretamente em arquivos HTML, o que torna mais fácil a criação de páginas web dinâmicas sem a necessidade de escrever grandes volumes de código de servlet.

É importante notar que JSP é frequentemente utilizado em conjunto com outras tecnologias Java EE como Servlets, JSTL (JavaServer Pages Standard Tag Library) e frameworks como Spring MVC para construir aplicações web.

Funcionamento Básico

Um arquivo JSP é essencialmente um documento HTML com trechos de código Java embutidos nele. Esses trechos de código são executados no servidor antes de a página ser enviada ao cliente. Quando um usuário faz uma solicitação para uma página JSP, o servidor web compila o arquivo JSP em um servlet. A partir daí, a execução é similar à de qualquer outro servlet, com o método service() sendo chamado para gerar a resposta ao cliente.

Veja um exemplo:

```
<html>
  <head>
    <title>
       <%="Hello World"%>
    </title>
  </head>
  <body>
       <%out.println("Hello Again!");%>
  </body>
</html>
```

Sintaxe e Tags

JSP utiliza uma série de tags especiais para embutir código Java em páginas HTML. Algumas das mais comuns são:

- <% ... %>: Para incluir código Java diretamente.
- <%= ... %>: Para inserir o valor de uma expressão Java na página.
- <%@ ... %>: Para diretivas que afetam todo o arquivo JSP.
- <jsp: ... >: Para elementos de ação que podem realizar tarefas como criar objetos ou redirecionar a resposta.

11225354498 - Maria Vitória Gonçalves Vieira

Objetos Implícitos

JSP tem um conjunto de "objetos implícitos" que podem ser referenciados no seu código a qualquer momento. A tabela abaixo dá um resumo desses objetos:

Objeto	Descrição
request	Objeto do tipo HttpServletRequest e contém a informação do pedido HTTP.
response	Objeto do tipo HttpServletResponse e contém a resposta HTTP que vai ser enviada ao cliente. Não é usado com frequência.
pageContext	Objeto do tipo PageContext e contém informações de contexto para execução da página.
application	Objeto do tipo ServletContext que permite compartilhar informações entre todos os componentes web da aplicação.
out	Objeto da classe JspWriter que permite imprimir para o response através do método println.
config	Objeto do tipo ServletConfig da página JSP.
page	Sinônimo do operador "this" do objeto HttpJspPage. É uma referência à página atual.
session	Objeto do tipo HttpSession que guarda informações da sessão de um usuário específico entre múltiplas requisições.
exception	É o objeto Throwable que é resultante de uma situação de erro numa página JSP.

Declarações definem variáveis, objetos e métodos.

```
//Declarações JSP

<%! public final static String[] estacoes = {"Primavera", "Verão", "Outono", "Inverno"} %>
```

Expressões: são utilizadas para embutir o resultado de uma expressão na Página JSP.

```
//Expressões JSP

<%= idade = idade + 1 %>
<%= "Esse é seu aniversário de " + idade + "anos" %>
```

Scriptlets são blocos de Código Java embutidos em uma Página JSP.

```
//Scriptlets JSP

<html>
    <body>
        <%! String mensagem = "Bem vindo!"; %>
        <% out.println(mensagem); %>
    </body>
</html>
```


11225354498 - Maria Vitória Gonçalves Vieira

Diretivas são utilizadas para importar classes de um pacote, inserir dados de arquivos externos e habilitar o uso de bibliotecas de tags.

- page define atributos de configuração da página JSP.
- include inclui recursos estáticos em uma Página JSP.
- taglib estende o conjunto de tags através de uma biblioteca de tags.

Vantagens

- Simplificação do Desenvolvimento: JSP torna mais fácil a criação de páginas dinâmicas, evitando a necessidade de escrever servlets extensos.
- Separação de Lógica e Apresentação: É mais simples manter o código relacionado à apresentação (HTML, CSS) separado da lógica de negócios (Java), o que é uma prática recomendada em desenvolvimento web.
- Reutilização de Componentes: JSP suporta tags customizadas e Expression Language (EL), permitindo a reutilização de componentes e simplificando o desenvolvimento.

Desafios e Limitações

- Performance: Embora a diferença seja geralmente mínima, JSP pode ser um pouco mais lento em comparação com servlets puros, pois o arquivo JSP precisa ser compilado em um servlet antes de ser executado.
- Complexidade: Para aplicações muito grandes e complexas, manter o código JSP pode se tornar desafiador, especialmente se as melhores práticas de design não forem seguidas.

JSF

JavaServer Faces (JSF) é um framework de desenvolvimento de interfaces de usuário para aplicações web Java. Foi introduzido como parte da plataforma Java EE (agora Jakarta EE) e tem como principal objetivo simplificar o desenvolvimento de interfaces de usuário baseadas em componentes.

Objetivos

1. Desenvolvimento Baseado em Componentes: JSF permite aos desenvolvedores criar aplicações web compostas por componentes reutilizáveis, como botões, campos de texto e tabelas, sem se preocupar com detalhes de baixo nível do ciclo de vida da solicitação e resposta HTTP.

11225354498 - Maria Vitória Gonçalves Vieira

2. Separação de Lógica e Apresentação: A arquitetura do JSF favorece a separação entre a lógica de negócios e a camada de apresentação. Isso facilita a manutenção e torna o código mais organizado.
3. Integração com Outras Tecnologias: JSF é frequentemente usado em combinação com outras tecnologias Java EE, como EJB (Enterprise JavaBeans) e JPA (Java Persistence API), para criar aplicações web.
4. Suporte a Internacionalização e Acessibilidade: JSF tem suporte nativo para recursos como internacionalização e acessibilidade, tornando-o adequado para aplicações globais e inclusivas.
5. Facilitar o Desenvolvimento: O framework visa simplificar tarefas comuns do desenvolvimento web, como validação de dados, navegação entre páginas e gerenciamento de estado.

O JSF é particularmente útil em cenários onde:

- É necessário construir uma interface de usuário rica e complexa com componentes reutilizáveis.
- A aplicação requer integração com outras tecnologias Java EE.
- A aplicação precisa ser altamente escalável e manutenível.

Partes do JSF

JSF possui várias partes essenciais que colaboram para fornecer uma experiência de desenvolvimento eficaz e produtiva. A seguir, detalharemos cada uma dessas partes:

Conjunto de Páginas Web

As páginas web no JSF geralmente são escritas em Facelets, que é uma linguagem de marcação baseada em XML. Estas páginas atuam como a camada de apresentação da aplicação, onde são colocados os componentes de interface de usuário.

Um conjunto de tags para adicionar componentes à página web

JSF fornece uma rica biblioteca de tags que podem ser usadas para adicionar diversos tipos de componentes à página, como campos de texto, botões, tabelas e outros elementos de formulário. As tags simplificam o processo de ligar componentes de interface de usuário a modelos de dados e de gerenciar eventos de usuário.

Um conjunto de Managed Beans (ou Beans Gerenciados)

11225354498 - Maria Vitória Gonçalves Vieira

Managed Beans são classes Java que contêm a lógica de negócios e gerenciam o estado da aplicação. Eles atuam como um intermediário entre a camada de apresentação e a camada de negócios, e podem ser facilmente acessados a partir de páginas Facelets através de expressões EL (Expression Language).

Um Descritor de Implantação Web (web.xml)

O arquivo web.xml é o arquivo de configuração padrão para aplicações web Java. Ele contém informações como o mapeamento de servlets, filtros e outros parâmetros de inicialização. Em JSF, este arquivo é frequentemente usado para definir configurações como o servlet de Faces e outras inicializações específicas do JSF.

Um ou mais arquivos de configuração (Ex: faces-config.xml)

O arquivo faces-config.xml é um arquivo de configuração específico do JSF onde você pode definir configurações como a navegação entre páginas, managed beans, regras de validação, etc. É um ponto central para gerenciar diversos aspectos do ciclo de vida da aplicação JSF.

Um conjunto de objetos customizados (Ex: validadores, conversores, etc)

JSF permite que desenvolvedores criem seus próprios validadores, conversores, e outros objetos customizados para estender a funcionalidade padrão. Estes objetos podem ser registrados no faces-config.xml e, em seguida, utilizados nas páginas Facelets.

Um conjunto de tags customizadas para representar objetos customizados

Além das tags padrão, o JSF também suporta tags customizadas que podem ser usadas para representar componentes ou funcionalidades customizadas. Essas tags permitem uma maior flexibilidade e reutilização de código, possibilitando aos desenvolvedores criar bibliotecas de componentes personalizadas.

Exemplo

A seguir está um exemplo simples de uma página JSF que contém um formulário para capturar o nome do usuário e exibir uma mensagem de boas-vindas. Este exemplo também utiliza um Managed Bean para armazenar e processar os dados.

Arquivo index.xhtml

11225354498 - Maria Vitória Gonçalves Vieira

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="http://java.sun.com/jsf/html">
<head>
    <title>Exemplo JSF</title>
</head>
<body>
    <h:form>
        <h:outputLabel for="nome" value="Digite seu nome:" />
        <h:inputText id="nome" value="#{userBean.nome}" />
        <h:commandButton value="Enviar" action="#{userBean.dizerOla}" />
    </h:form>
    <h:outputText value="#{userBean.mensagem}" />
</body>
</html>
```

Arquivo UserBean.java

11225354498 - Maria Vitória Gonçalves Vieira

```java
import javax.faces.bean.ManagedBean;
import javax.faces.bean.SessionScoped;

@ManagedBean
@SessionScoped
public class UserBean {
    private String nome;
    private String mensagem;

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public String getMensagem() {
        return mensagem;
    }
```


11225354498 - Maria Vitória Gonçalves Vieira

```
    public void setMensagem(String mensagem) {
        this.mensagem = mensagem;
    }

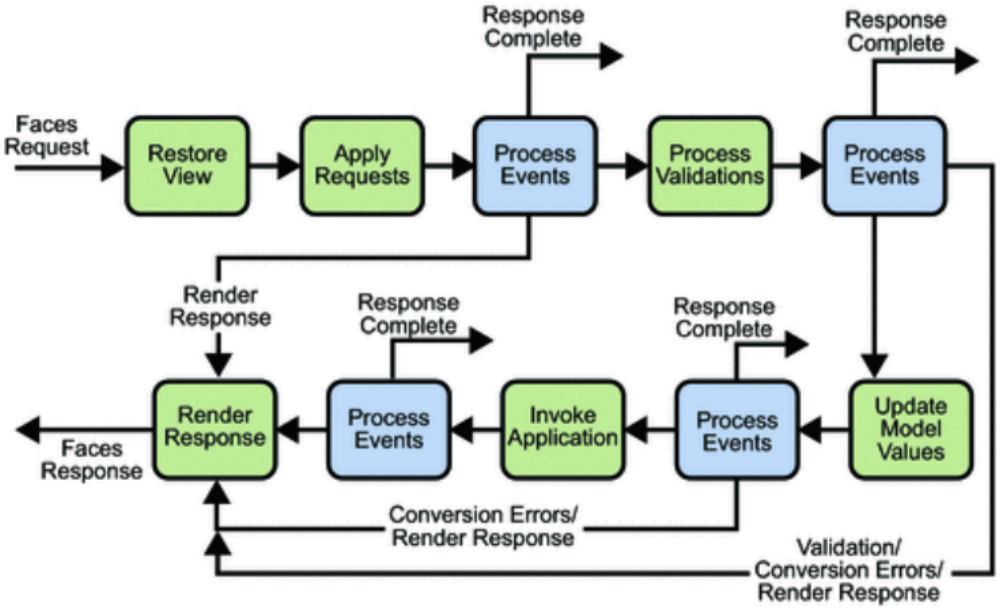
    public String dizerOla() {
        if (nome != null && nome.length() > 0) {
            mensagem = "Olá, " + nome + "!";
        } else {
            mensagem = "Olá, usuário anônimo!";
        }
        return null;  // Permanece na mesma página
    }
}
```

Neste exemplo, a página index.xhtml contém um formulário que utiliza o Managed Bean UserBean para armazenar o nome inserido pelo usuário. Quando o usuário clica no botão "Enviar", o método dizerOla é invocado para gerar uma mensagem de boas-vindas que é então exibida na mesma página.

Ciclo de Vida

Veja o ciclo de vida de requisição e resposta de uma página JSF:

11225354498 - Maria Vitória Gonçalves Vieira



- Restore View: se for a primeira visita à página, deve-se criar a visão. Se não for a primeira vez, os objetos e estruturas de dados que representam a visão são restaurados.
- Apply Request Values: qualquer dado que for enviado como parte da requisição é passado para os componentes apropriados. Essas visões atualizam seus estados com os valores dos dados.
- Process Validation: os dados que foram submetidos com o formulário são validados (se já não o foram anteriormente). Assim como na fase anterior, isso ainda não atualiza os objetos de negócio na aplicação. Isso ocorre porque, se a Aplicação atualizar os objetos de negócio junto com a validação dos dados e uma parte da validação falhar, o modelo será atualizado com um estado inválido.
- Update Model Values: após todas essas validações terminarem, os objetos de negócio que criam a aplicação são atualizados com os dados validados da requisição. Se qualquer um dos dados precisar ser convertido em um formato diferente para atualizar o modelo (Ex: String para Data), a conversão ocorrerá nessa fase.
- Invoke Application: durante essa fase, os métodos de ação de qualquer botão ou link que foi ativado serão chamados. Além disso, todos os eventos que foram gerados durante as fases anteriores e que ainda não tenham sido manipulados são passados para a Aplicação Web para que ela possa concluir qualquer outro processamento da requisição que seja necessário.
- Render Response: os Componentes UI de resposta são renderizados e a resposta é enviada para o cliente. O estado dos componentes é salvo de modo que a árvore de componente possa ser restaurada quando o cliente enviar outra requisição.

11225354498 - Maria Vitória Gonçalves Vieira

JPA

Java Persistence API (JPA) é uma especificação do Java Enterprise Edition (Java EE) que visa fornecer uma abordagem padronizada para o mapeamento objeto-relacional (ORM) em aplicações Java. Ela serve como uma ponte entre modelos de dados orientados a objetos e modelos de dados relacionais, como os usados em bancos de dados. Ao fornecer um conjunto de APIs e anotações, a JPA permite que os desenvolvedores persistam, recuperem, atualizem e excluam objetos Java de forma eficiente em bancos de dados relacionais, sem a necessidade de escrever código SQL "cru".

Objetivos

1. Abstração sobre SQL: A JPA permite que os desenvolvedores trabalhem diretamente com objetos Java, evitando a necessidade de escrever consultas SQL para interagir com o banco de dados.
2. Portabilidade: Como uma especificação, a JPA é implementada por diversos fornecedores, como Hibernate, EclipseLink e OpenJPA. Isso torna o código baseado em JPA portátil entre diferentes plataformas e bancos de dados.
3. Flexibilidade e Manutenção: A JPA facilita a manutenção de código e torna o desenvolvimento mais rápido, já que menos código de "boilerplate" (código secundário) é necessário.
4. Conjunto de Funcionalidades: Suporte para consultas complexas, relacionamentos entre entidades, herança, polimorfismo e outras características avançadas de modelagem de dados.
5. Integração com Outras Tecnologias Java EE: A JPA é frequentemente usada em conjunto com outras especificações Java EE, como EJB (Enterprise JavaBeans), JSF (JavaServer Faces) e JAX-RS (Java API for RESTful Web Services), proporcionando um ecossistema coeso para o desenvolvimento de aplicações empresariais.

Uso

JPA é comumente usada em aplicações que necessitam de interação robusta e eficiente com bancos de dados. Ela é adequada tanto para aplicações web quanto para aplicações desktop. Alguns cenários típicos de uso são: Sistemas de Gestão Empresarial (ERP), Aplicações de Comércio Eletrônico, Sistemas de Informação Geográfica (GIS), Redes Sociais e Aplicações Bancárias e Financeiras

JDBC versus JPA

11225354498 - Maria Vitória Gonçalves Vieira

Java Database Connectivity (JDBC) e a Java Persistence API (JPA) são ambas tecnologias Java destinadas à interação com bancos de dados, mas elas são usadas para propósitos e em níveis de abstração diferentes. A tabela abaixo apresenta as principais diferenças:

Critério	JDBC	JPA
Nível de Abstração	**Baixo: SQL direto e manipulação de registros**	**Alto: Mapeamento objeto-relacional (ORM)**
Portabilidade	**Parcial: SQL pode ser específico do banco de dados**	**Alta: Abstrai especificidades de bancos de dados**
Facilidade de Uso	**Menor: Requer mais código para tarefas simples**	**Maior: Oferece funcionalidades como persistência automática**
Flexibilidade	**Alta: Controle granular sobre SQL e transações**	**Moderada: Menos controle direto, mas suporta otimizações**
Manutenção	**Desafiadora: Mistura de lógica de negócios e SQL**	**Facilitada: Separação entre lógica de negócios e acesso a dados**
Escalabilidade	**Pode ser complexo devido à mistura de SQL e lógica de negócios**	**Geralmente mais fácil de escalar**
Gerenciamento de Transações	**Manual: Implementação cuidadosa requerida**	**Integrado: Mais fácil e seguro, especialmente com componentes Java EE**

Ecossistema JPA

O JPA é composto por várias partes e componentes. Vamos detalhar esse aspecto a seguir.

11225354498 - Maria Vitória Gonçalves Vieira

Java Persistence API (JPA)

Objetivo: O JPA visa facilitar o mapeamento entre objetos Java e tabelas de banco de dados relacional. Sua principal finalidade é o gerenciamento de dados relacionais em aplicações Java.

Funcionamento: Utiliza uma abordagem de Mapeamento Objeto-Relacional (ORM) para abstrair a complexidade do SQL, permitindo que os desenvolvedores se concentrem na lógica do negócio.

Características:

- Oferece anotações para mapear classes Java às tabelas de banco de dados.
- Oferece serviços de persistência automáticos como inserção, atualização, exclusão e consulta.

Java Persistence Query Language (JPQL)

Objetivo: Fornecer uma linguagem de consulta orientada a objetos para realizar operações de busca em entidades persistentes.

Funcionamento: JPQL é uma extensão da SQL, projetada para funcionar com o modelo de objetos da JPA. É tipicamente mais abstrata que SQL e se concentra em objetos e seus relacionamentos.

Características:

- As consultas são geralmente estáticas e definidas em tempo de desenvolvimento.
- Permite consultas polimórficas e navegação em relações entre entidades.

Java Persistence Criteria API (JPCA)

Objetivo: Permitir a criação de queries dinâmicas através de uma abordagem programática.

Funcionamento: Utiliza classes e métodos Java específicos para construir queries em tempo de execução. É particularmente útil quando as consultas são complexas e construídas dinamicamente com base em vários parâmetros.

Características:

- Fornecimento de um tipo seguro para criação de queries.

11225354498 - Maria Vitória Gonçalves Vieira

- É mais fácil de manter e ajustar dinamicamente as consultas.

Mapeamento de Metadados O/R

Objetivo: Este componente visa estabelecer a correspondência entre os objetos Java e as tabelas e colunas do banco de dados.

Funcionamento: O mapeamento pode ser realizado tanto através de descritores de implantação em arquivos XML quanto por meio de anotações Java.

Características Notáveis:

- Os descritores de implantação XML permitem um mapeamento externo que não afeta o código fonte.
- As Anotações são incorporadas diretamente no código e tornam o mapeamento mais transparente, mas podem tornar o código mais acoplado.

Entidades e Anotações

Entidades

Entidades são classes Java POJO (Plain Old Java Object) que representam tabelas de banco de dados em um ambiente JPA. Uma entidade representa uma linha da tabela e cada instância da entidade corresponde a um registro único no banco de dados.

Características:

- Estado Persistente: As propriedades da classe, geralmente mapeadas para colunas do banco de dados, mantêm o estado da entidade.
- Identidade: Cada entidade tem uma chave primária que a identifica de forma única.
- Ciclo de Vida: As entidades têm estados bem definidos, como "novo", "gerenciado", "desanexado" e "removido".

Uso Comum:

- Representar o modelo de domínio em uma aplicação.
- Facilitar o CRUD (Criar, Ler, Atualizar, Excluir) de registros no banco de dados.

Anotações

11225354498 - Maria Vitória Gonçalves Vieira

Anotações são metadados que fornecem informações adicionais ao compilador e ao JPA sobre o comportamento das classes, métodos e campos. No contexto da JPA, as anotações são usadas para definir o mapeamento entre as entidades e as tabelas do banco de dados.

Características:

- Facilidade de Uso: As anotações tornam o código autoexplicativo e mais fácil de manter.
- Configuração Declarativa: Ao invés de usar arquivos de configuração externos (como XML), o mapeamento é feito diretamente no código.
- Personalização: Algumas anotações permitem um alto grau de personalização, como estratégias de herança, mapeamento de relações complexas entre entidades e conversões de tipos personalizadas.

Veja as anotações mais comuns do JPA:

ANOTAÇÃO	DESCRIÇÃO
@Entity	Indica que uma classe é uma entidade que deve ser persistida como tabela.
@Table	Indica o nome de uma tabela para qual a entidade é mapeada.
@Column	Indica que um atributo de uma classe é uma coluna de uma tabela.
@Id	Indica que um atributo é a chave primária de uma entidade.
@namedQuery	Define uma consulta.
@namedQueries	Define várias consultas.
@UniqueConstraint	Indica que uma propriedade não pode conter valores duplicados.
@Transient	Indica que uma propriedade não deve ser persistida no banco de dados.
@Inheritance	Define um relacionamento de herança à entidade.
@ManyToOne	Mapeamento muitos para um (N:I).
@OneToMany	Mapeamento um para muitos (I:N).
@OneToOne	Mapeamento um para um (I:I).

Agora veja um exemplo de código que mostra uma entidade e suas anotações:

11225354498 - Maria Vitória Gonçalves Vieira

```java
@Entity
@Table(name = "usuarios")
public class Usuario {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "nome")
    private String nome;

    // Getters e Setters
}
```

Entity Manager

O EntityManager é uma interface fundamental no JPA, responsável por fornecer as operações CRUD (Criar, Ler, Atualizar, Deletar) para entidades. Ele atua como um ponto central para interagir com a camada de persistência em aplicações Java.

Objetivos:

1. Gerenciamento de Entidades: Ele gerencia o ciclo de vida das entidades desde a criação até a remoção.
2. Persistência e Recuperação: Oferece métodos para persistir novas entidades e recuperar entidades existentes do banco de dados.
3. Consulta: Fornece mecanismos para executar consultas ao banco de dados, tanto estáticas (JPQL) como dinâmicas (Criteria API).
4. Transacionalidade: Lida com a gerência de transações, permitindo que as operações sejam atômicas, consistentes, isoladas e duráveis (ACID).

Principais Métodos

- persist(Object entity): Persiste uma nova entidade no banco de dados.
- find(Class<T> entityClass, Object primaryKey): Encontra uma entidade pelo tipo e chave primária.
- remove(Object entity): Remove uma entidade do banco de dados.
- merge(T entity): Atualiza uma entidade destacada, sincronizando-a com o banco de dados.

11225354498 - Maria Vitória Gonçalves Vieira

- detach(Object entity): Destaca uma entidade, fazendo com que as alterações subsequentes não sejam automaticamente persistidas.

Exemplos de uso:

```
// Criando uma nova entidade
EntityManager em = entityManagerFactory.createEntityManager();
em.getTransaction().begin();
Usuario usuario = new Usuario("nome", "email");
em.persist(usuario);
em.getTransaction().commit();

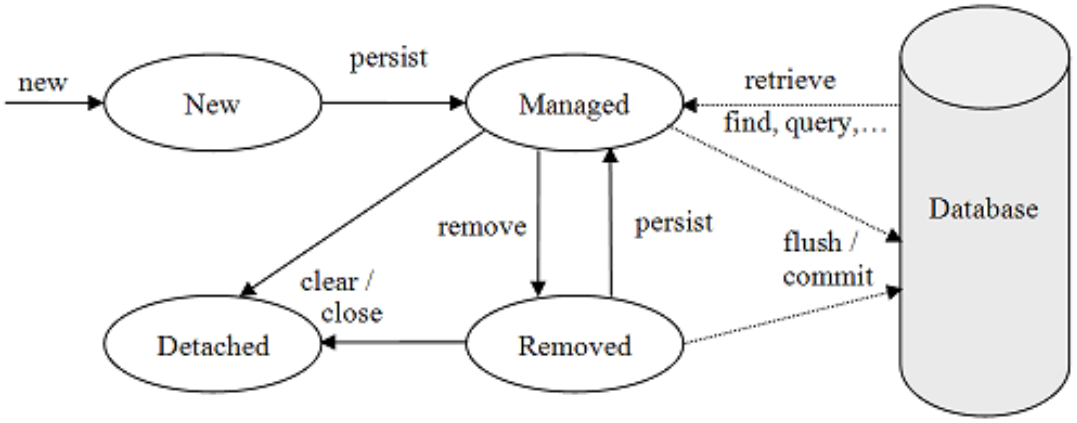
// Buscando uma entidade existente
Usuario usuarioExistente = em.find(Usuario.class, 1);

// Atualizando uma entidade
em.getTransaction().begin();
usuarioExistente.setEmail("novo_email@email.com");
em.getTransaction().commit();

// Removendo uma entidade
em.getTransaction().begin();
em.remove(usuarioExistente);
em.getTransaction().commit();
```

Ciclo de Vida de Entidades

11225354498 - Maria Vitória Gonçalves Vieira



Novo (New)

O estado "Novo" refere-se à fase inicial da vida de uma entidade JPA. Um objeto neste estado foi instanciado, mas ainda não foi associado a um EntityManager.

Características:

- Não Sincronizado: Qualquer dado contido no objeto não é reconhecido pelo EntityManager.
- Transição: Pode ser movido para o estado "Gerenciado" usando o método persist() do EntityManager.

```
Usuario novoUsuario = new Usuario(); // Estado Novo
```

Gerenciado (Managed)

O estado "Gerenciado" ocorre quando uma entidade é persistida no banco de dados por meio do EntityManager. Entidades que são recuperadas do banco de dados pelo EntityManager também entram neste estado.

Características:

- Sincronizado: Quaisquer mudanças feitas na entidade serão automaticamente propagadas para o banco de dados.
- Gestão Automática: O EntityManager controla automaticamente as operações de ciclo de vida da entidade.

```
entityManager.persist(novoUsuario); // Estado Gerenciado
```


11225354498 - Maria Vitória Gonçalves Vieira

Removido (Removed)

Neste estado, um objeto que inicialmente estava em estado "Gerenciado" tem seu estado alterado para "Removido", resultando em sua exclusão do banco de dados.

Características:

- Exclusão: O objeto será excluído do banco de dados no momento em que a transação for concluída.
- Desassociação: O objeto não é mais considerado para operações de persistência subsequentes.

```
entityManager.remove(usuarioGerenciado); // Estado Removido
```

Destacado (Detached)

O estado "Destacado" representa um objeto que foi desconectado do EntityManager.

Características

- Não Sincronizado: Alterações feitas na entidade não serão refletidas no banco de dados a menos que a entidade seja novamente associada com um EntityManager.
- Reanexação: A entidade pode ser reanexada usando métodos como merge().

```
entityManager.detach(usuarioGerenciado); // Estado Destacado
```

EJB

Enterprise JavaBeans (EJB) é uma especificação de servidor para a plataforma Java EE que permite a criação de componentes escaláveis, transacionais e distribuídos. O EJB é usado principalmente para desenvolver aplicações corporativas grandes e robustas.

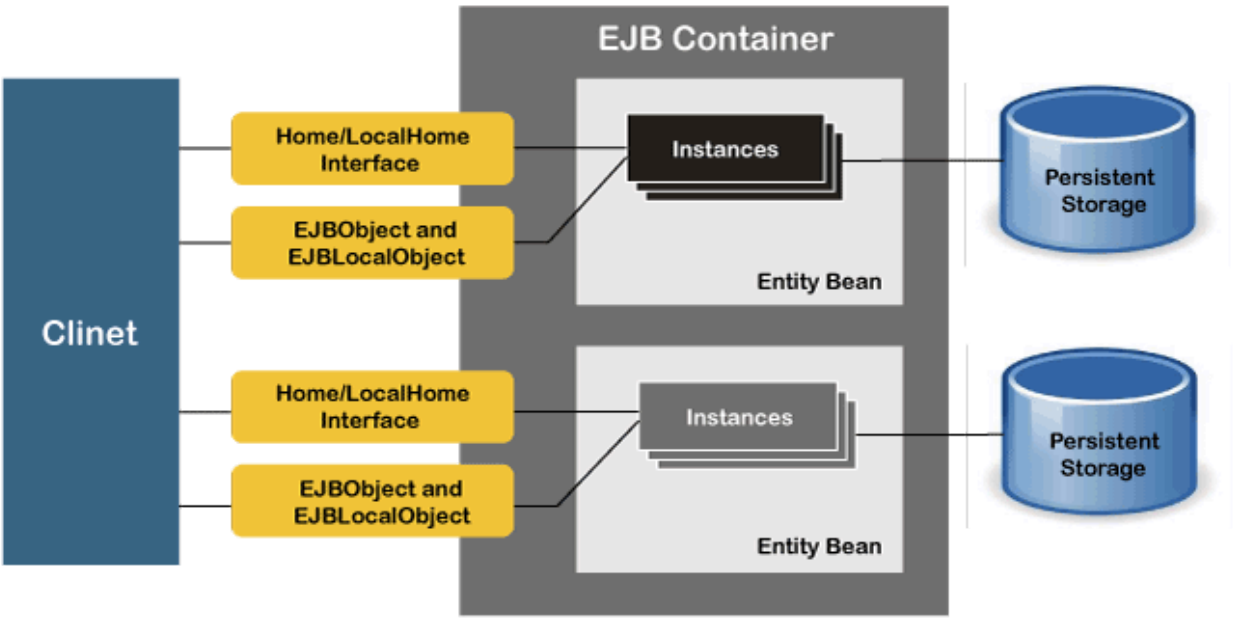
Veja alguns de seus objetivos:

11225354498 - Maria Vitória Gonçalves Vieira

1. Desacoplamento de Lógica de Negócios: Os EJBs oferecem um ambiente onde a lógica de negócios pode ser desacoplada da lógica de apresentação e persistência, tornando a aplicação mais modular e fácil de manter.
2. Gerenciamento de Transações: Fornecem um modelo declarativo para gerenciamento de transações, permitindo que o desenvolvedor se concentre na lógica de negócios em vez de na sincronização de recursos.
3. Segurança: Através de mecanismos como autenticação e autorização, EJBs garantem um ambiente seguro para executar operações sensíveis.
4. Escalabilidade e Desempenho: Eles são projetados para serem escaláveis, aproveitando técnicas como o pooling de instâncias e o balanceamento de carga.
5. Facilidade de Integração: EJBs podem ser facilmente integrados com outras tecnologias Java EE como JPA, JMS, JTA, e muitas outras.

Container EJB

O Container EJB (Enterprise JavaBeans) é uma componente fundamental em aplicações empresariais desenvolvidas com Java EE. Atua como uma camada intermediária entre o código do bean e o sistema operacional, fornecendo uma série de serviços que auxiliam no desenvolvimento, tornando-o mais produtivo e seguro. Veja os principais serviços oferecidos pelo container EJB:



Ciclo de Vida

O Container EJB gerencia o ciclo de vida completo dos Enterprise Beans. Isso significa que o desenvolvedor não precisa implementar código para criar, inicializar, ativar ou destruir beans. O container cuida de todas essas operações de forma automática, o que simplifica bastante o código e reduz a margem de erro.

Segurança

Um dos pilares das aplicações empresariais é a segurança, e o Container EJB oferece suporte robusto para autenticação e autorização. O controle de acesso é orientado a papéis (roles), permitindo uma segregação de funções mais eficiente. Isso possibilita que apenas usuários autorizados tenham acesso a funcionalidades específicas, ajudando a manter a integridade e a confidencialidade dos dados.

Transações

O Container EJB é responsável por gerenciar transações, o que é vital em aplicações empresariais onde a consistência de dados é crítica. Ele inicia, coordena e finaliza transações de forma automática. Isso permite que os desenvolvedores se concentrem na lógica de negócios, enquanto o container cuida dos aspectos técnicos de garantir que as transações sejam processadas de forma segura e eficaz.

Persistência

Em aplicações empresariais, geralmente é necessário persistir dados em um banco de dados. O Container EJB alivia o desenvolvedor da necessidade de escrever código específico para essa finalidade. Isso é particularmente útil quando se utiliza o JPA (Java Persistence API), que funciona de forma transparente com o EJB para fornecer persistência de dados.

Estado

Para aplicações que exigem manutenção de estado conversacional entre as interações do cliente, o Container EJB oferece mecanismos automáticos para salvar e recuperar esse estado. Isso é particularmente útil para Stateful Session Beans, que necessitam manter informações entre várias solicitações do mesmo cliente.

Tipos de Beans

Existem vários tipos de beans, cada um com seus próprios objetivos e casos de uso. A seguir, vamos ver os tipos mais comuns:

11225354498 - Maria Vitória Gonçalves Vieira

Session Beans

- Stateless Session Beans: São beans que não mantêm estado entre invocações de método. Úteis para operações que são independentes e que não requerem informação de contexto entre chamadas.
- Stateful Session Beans: Mantêm estado entre invocações de método e são associados a um único cliente durante o ciclo de vida do bean. Úteis para cenários que requerem um fluxo conversacional com o cliente.
- Singleton Session Beans: São beans que existem em uma única instância por aplicação e são usados para compartilhar estado ou recursos entre vários clientes.

Message-Driven Beans (MDB)

- Message-Driven Beans: São beans orientados por mensagens e ativados por mensagens de um tópico ou fila. São completamente desacoplados, permitindo processamento assíncrono e escalável.

Managed Beans

- Managed Beans: São beans POJO (Plain Old Java Object) que podem ser gerenciados pelo contêiner CDI (Contexts and Dependency Injection). Eles são usados principalmente em aplicações web e podem ser de diferentes escopos, como request, session e application.

Enterprise JavaBeans (EJB)

- Como já mencionado anteriormente, os EJBs (Stateless, Stateful e Singleton) são beans empresariais que fornecem serviços como transações e segurança.

CDI Beans

- CDI Beans: São beans gerenciados pelo CDI e podem ser qualquer tipo de bean que suporte as anotações do CDI. Eles são muito flexíveis e podem ser usados em diversas partes da aplicação, desde a camada de apresentação até a camada de negócios.

Bean de Acesso a Dados

- Entity Beans: Embora considerados obsoletos em versões mais modernas do EJB, eles são usados para encapsular a lógica de acesso a dados em aplicações mais antigas.

11225354498 - Maria Vitória Gonçalves Vieira

Cada um desses Beans pode ser "anotado" no código, indicando ao ambiente de execução sobre qual tipo de bean se trata naquele momento. Veja as anotações comuns:

- Stateless: @Stateless
- Stateful: @Stateful
- Singleton: @Singleton
- Message-Driven: @MessageDriven
- Managed Bean: @ManagedBean
- CDI: @Named

Veja um exemplo de stateless bean:

```
@Stateless
public class OrderServiceBean implements OrderService {
    @PersistenceContext(unitName = "OrderPU")
    EntityManager em;

    public void createOrder(Order order) {
        em.persist(order);
    }

    public Order findOrder(Long id) {
        return em.find(Order.class, id);
    }
}
```

Inversão de Controle e Injeção de Dependência no Contexto de EJB

Inversão de Controle (IoC)

Inversão de Controle (IoC) é um padrão de projeto de software que se refere à transferência da responsabilidade de controle do fluxo da aplicação do próprio código para um componente externo, como o Container EJB em um ambiente Java EE. O objetivo é modularizar a aplicação e torná-la mais extensível, manutenível e testável. No contexto de EJB, o Container EJB gerencia o ciclo de vida dos beans, segurança,

transações e outros aspectos, permitindo que os desenvolvedores se concentrem na lógica de negócios.

Injeção de Dependência

Injeção de Dependência (DI) é uma forma de IoC onde as dependências de um objeto (como serviços ou recursos) são fornecidas por um agente externo, em vez de serem criadas pelo próprio objeto. No contexto de EJB, o Container EJB automaticamente injeta as dependências necessárias nos beans no momento da criação. Este processo facilita a reutilização, teste e manutenção do código.

Como Funciona no Contexto de EJB

1. Configuração: Primeiro, as dependências e os beans são configurados, seja através de anotações diretamente no código ou através de arquivos XML.
2. Instanciação: Quando um EJB é necessário, o Container EJB cria uma nova instância ou recupera uma do pool existente.
3. Injeção: Antes de disponibilizar o bean para uso, o Container EJB injeta todas as dependências declaradas. Isso pode incluir recursos como um DataSource, outros EJBs, serviços de mensageria, etc.
4. Gerenciamento de Ciclo de Vida: Após a injeção, o Container EJB gerencia o ciclo de vida do bean, fornecendo serviços como gerenciamento de transações e segurança.
5. Desacoplamento: A injeção de dependência e a inversão de controle ajudam a desacoplar os componentes de uma aplicação, tornando mais fácil alterar a implementação de uma dependência sem afetar os componentes que dependem dela.

Exemplo

Primeiro, vamos criar uma interface para o serviço que desejamos injetar.

```
public interface GreetingService {
    String greet(String name);
}
```

Agora, vamos implementar o serviço.

11225354498 - Maria Vitória Gonçalves Vieira

```java
import javax.ejb.Stateless;

@Stateless
public class GreetingServiceImpl implements GreetingService {
    @Override
    public String greet(String name) {
        return "Hello, " + name;
    }
}
```

Neste exemplo, o uso da anotação @Stateless indica que este bean não mantém estado entre as chamadas de método e pode ser gerenciado pelo Container EJB.

A seguir, um bean que consome o serviço GreetingService:

```java
import javax.ejb.Stateless;
import javax.inject.Inject;

@Stateless
public class HelloWorldBean {

    @Inject
    private GreetingService greetingService;

    public void sayHello(String name) {
        System.out.println(greetingService.greet(name));
    }
}
```

Aqui, usamos a anotação @Inject para injetar uma instância de GreetingService. O Container EJB cuida da criação e gerenciamento desta instância.

Agora executando:

11225354498 - Maria Vitória Gonçalves Vieira

```java
import javax.ejb.EJB;

public class Main {

    @EJB
    private static HelloWorldBean helloWorldBean;

    public static void main(String[] args) {
        helloWorldBean.sayHello("John");
    }
}
```

Neste exemplo, a anotação @EJB é usada para injetar uma instância do HelloWorldBean no ponto de entrada da aplicação. O Container EJB gerencia o ciclo de vida e as dependências de HelloWorldBean.

Com este conjunto de código, você pode observar que a Inversão de Controle (IoC) e a Injeção de Dependência (DI) estão sendo aplicadas. O Container EJB cria e injeta as instâncias necessárias automaticamente, permitindo que o desenvolvedor se concentre na lógica de negócios.

JTA

A Java Transaction API (JTA) é uma especificação em Java que permite aos aplicativos executarem transações de forma distribuída e coordenada. Ou seja, ela fornece uma maneira de agrupar várias operações em um único bloco de trabalho, assegurando que todas sejam concluídas com sucesso ou que nenhum efeito colateral seja deixado em caso de falha.

Objetivos

1. Atomicidade: Garantir que todas as operações de uma transação sejam concluídas com sucesso ou que todas sejam revertidas em caso de falha, mantendo a integridade dos dados.

11225354498 - Maria Vitória Gonçalves Vieira

2. Consistência: Certificar que os dados permaneçam consistentes antes e depois da transação.
3. Isolamento: Assegurar que as transações sejam independentes umas das outras.
4. Durabilidade: Certificar-se de que as mudanças de uma transação bem-sucedida sejam permanentes.
5. Abstração: Fornecer uma interface de alto nível para gerenciamento de transações, ocultando detalhes complexos de implementação.
6. Gerenciamento de Recursos Distribuídos: Facilitar o gerenciamento de transações que envolvem múltiplos recursos, como diferentes bases de dados ou sistemas de mensageria.

Uso

JTA é geralmente utilizada em aplicações corporativas complexas que requerem transações envolvendo múltiplos recursos distribuídos. Ela é amplamente adotada em combinação com outras tecnologias Java EE como EJB (Enterprise JavaBeans), JPA (Java Persistence API) e JMS (Java Message Service).

Exemplo de uso:

11225354498 - Maria Vitória Gonçalves Vieira

```java
public class MyTransactionBean {

    @Inject
    private UserTransaction userTransaction;

    @Inject
    private EntityManager entityManager;

    public void performTransaction() {
        try {
            userTransaction.begin();

            // Operações de negócios e persistência
            // Ex: entityManager.persist(someEntity);

            userTransaction.commit();
        } catch (Exception e) {
            try {
                userTransaction.rollback();
            } catch (Exception rollbackEx) {
                // Tratar falha no rollback
            }
            // Tratar falha na transação
        }
    }
}
```

Neste exemplo, UserTransaction é uma interface da JTA que permite demarcar transações manualmente. O objeto EntityManager é da JPA e é usado para operações de persistência. O método performTransaction ilustra como uma transação típica pode ser executada e gerenciada usando JTA.

O uso da JTA libera o desenvolvedor de gerenciar manualmente os detalhes complexos de transações, como commit, rollback e recuperação, permitindo que ele se concentre na lógica de negócios do aplicativo.

Anotações

Anotações

JTA fornece anotações para demarcar transações nos métodos de um Enterprise Java Bean (EJB), facilitando o uso de transações em um ambiente gerenciado. Veja as principais:

- @Transactional: Usada para demarcar que um método ou classe deve ser executado dentro de uma transação.
- @TransactionAttribute: Permite especificar o tipo de atributo de transação em métodos EJB. Por exemplo, TransactionAttributeType.REQUIRED indica que um método precisa de uma transação em andamento para ser executado.
- @TransactionManagement: Utilizada em um EJB para especificar se o container ou a aplicação será responsável pelo gerenciamento da transação.

JTA e 2PC

O protocolo Two-Phase Commit (2PC) é um padrão de projeto para garantir transações distribuídas atômicas. No contexto do JTA (Java Transaction API), você pode usar 2PC para assegurar que múltiplos recursos sejam coordenados para completar uma transação de forma coerente. Vejamos um exemplo simples de como realizar um Two-Phase Commit com JTA.

Ambiente

Suponhamos que temos duas bases de dados diferentes, database1 e database2, e queremos atualizar ambas de forma atômica.

Definição do recurso de transação no context.xml ou diretamente na aplicação:

```
<Resource name="jdbc/Database1" .../>
<Resource name="jdbc/Database2" .../>
```

Código Java usando JTA para 2PC:

```java
import javax.annotation.Resource;
import javax.transaction.UserTransaction;
import javax.sql.DataSource;
import java.sql.Connection;
import java.sql.PreparedStatement;

public class TwoPhaseCommitExample {

    @Resource(lookup = "java:jboss/UserTransaction")
    private UserTransaction userTransaction;

    @Resource(lookup = "java:comp/env/jdbc/Database1")
    private DataSource dataSource1;

    @Resource(lookup = "java:comp/env/jdbc/Database2")
    private DataSource dataSource2;

    public void performTransaction() {
        Connection conn1 = null;
        Connection conn2 = null;
        PreparedStatement pstmt1 = null;
        PreparedStatement pstmt2 = null;

        try {
            // Iniciar transação
            userTransaction.begin();
```


11225354498 - Maria Vitória Gonçalves Vieira

```
            // Conectar e realizar operações em Database1
            conn1 = dataSource1.getConnection();
            pstmt1 = conn1.prepareStatement("UPDATE table1 SET column1 = ? WHERE
column2 = ?");
            pstmt1.setString(1, "new_value1");
            pstmt1.setString(2, "value2");
            pstmt1.executeUpdate();

            // Conectar e realizar operações em Database2
            conn2 = dataSource2.getConnection();
            pstmt2 = conn2.prepareStatement("UPDATE table2 SET columnA = ? WHERE
columnB = ?");
            pstmt2.setString(1, "new_valueA");
            pstmt2.setString(2, "valueB");
            pstmt2.executeUpdate();

            // Commit da transação - Two-Phase Commit ocorre aqui
            userTransaction.commit();
        } catch (Exception e) {
            e.printStackTrace();
            try {
                // Rollback em caso de falha
                userTransaction.rollback();
            } catch (Exception rbEx) {
                rbEx.printStackTrace();
            }
```

```
        } finally {
            try {
                if (pstmt1 != null) pstmt1.close();
                if (pstmt2 != null) pstmt2.close();
                if (conn1 != null) conn1.close();
                if (conn2 != null) conn2.close();
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    }
}
```

Explicação

1. Begin Transaction: A transação é iniciada usando UserTransaction.begin().
2. Database Operations: Ambas as operações de atualização são preparadas, mas ainda não comprometidas.
3. Commit: Usando UserTransaction.commit(), a fase de "prepare" do Two-Phase Commit é iniciada. Se ambas as bases de dados responderem que a operação pode ser realizada, então a fase de "commit" é iniciada, tornando todas as operações permanentes.
4. Rollback: Em caso de falha, a transação é revertida em todos os recursos envolvidos, garantindo a consistência do sistema.

JMS

Java Message Service (JMS) é uma API do Java EE que fornece um mecanismo para comunicação assíncrona entre componentes de uma aplicação ou entre diferentes aplicações distribuídas. A comunicação assíncrona através de JMS permite que os

componentes do sistema interajam sem estarem simultaneamente ativos ou disponíveis. Vamos explorar alguns conceitos essenciais relacionados a mensagens assíncronas no contexto do JMS.

Tipos de Mensagens

1. TextMessage: Mensagens de texto simples, geralmente contendo dados em formato XML, JSON ou simplesmente texto.
2. BytesMessage: Um fluxo de bytes sem formatação.
3. MapMessage: Um conjunto de pares chave-valor.
4. ObjectMessage: Um objeto serializável em Java.
5. StreamMessage: Um fluxo de valores primitivos em Java.

Componentes Principais

1. Producers: São responsáveis por criar e enviar mensagens para um destino.
2. Consumers: São responsáveis por receber mensagens de um destino.
3. Destinations: Podem ser Topics (para publicação e subscrição) ou Queues (para ponto a ponto).

Modelos de Comunicação

1. Point-to-Point (P2P): Utiliza o conceito de queue. A mensagem enviada por um producer para uma queue específica é consumida por um único consumer.
2. Publish-Subscribe (Pub-Sub): Utiliza o conceito de topic. Uma mensagem enviada a um topic pode ser consumida por vários subscribers.

Qualidades de Serviço

1. At-Most-Once: A mensagem é entregue no máximo uma vez, e a entrega não é garantida.
2. At-Least-Once: A mensagem é entregue pelo menos uma vez, e pode ser entregue mais de uma vez.
3. Exactly-Once: Garante que a mensagem será entregue exatamente uma vez.

Transações

O JMS suporta transações, permitindo agrupar várias operações de envio e/ou recepção em uma única unidade de trabalho atômica.

Persistência

11225354498 - Maria Vitória Gonçalves Vieira

Mensagens podem ser persistentes ou não-persistentes. Mensagens persistentes são armazenadas em um armazenamento estável e sobrevivem a falhas no sistema, enquanto mensagens não-persistentes podem ser perdidas em caso de falhas.

Filtro e Seleção

Os consumers podem usar selectors para filtrar mensagens com base em critérios específicos definidos nos cabeçalhos ou propriedades das mensagens.

Exemplo

Veja um exemplo simplificado de código usando JMS para enviar e receber mensagens em um modelo de fila ponto a ponto (Point-to-Point).

Produtor de Mensagens

```
import javax.jms.Connection;

import javax.jms.ConnectionFactory;

import javax.jms.MessageProducer;

import javax.jms.Queue;

import javax.jms.Session;

import javax.jms.TextMessage;

import javax.naming.InitialContext;



public class MessageProducerExample {

    public static void main(String[] args) throws Exception {

        InitialContext context = new InitialContext();

        ConnectionFactory factory = (ConnectionFactory)
context.lookup("jms/ConnectionFactory");

        Connection connection = factory.createConnection();



        Session session = connection.createSession(false,
Session.AUTO_ACKNOWLEDGE);
```

```
        Queue queue = (Queue) context.lookup("jms/Queue");


        MessageProducer producer = session.createProducer(queue);


        TextMessage message = session.createTextMessage("Hello, JMS!");
        producer.send(message);


        session.close();
        connection.close();
        context.close();
    }
}
```

Consumidor de Mensagens

```
import javax.jms.Connection;
import javax.jms.ConnectionFactory;
import javax.jms.Message;
import javax.jms.MessageConsumer;
import javax.jms.Queue;
import javax.jms.Session;
import javax.jms.TextMessage;
import javax.naming.InitialContext;


public class MessageConsumerExample {
    public static void main(String[] args) throws Exception {
```

```
        InitialContext context = new InitialContext();

        ConnectionFactory factory = (ConnectionFactory)
context.lookup("jms/ConnectionFactory");

        Connection connection = factory.createConnection();



        Session session = connection.createSession(false,
Session.AUTO_ACKNOWLEDGE);

        Queue queue = (Queue) context.lookup("jms/Queue");



        MessageConsumer consumer = session.createConsumer(queue);



        connection.start();



        Message message = consumer.receive();



        if (message instanceof TextMessage) {

            TextMessage textMessage = (TextMessage) message;

            System.out.println("Received Message: " + textMessage.getText());

        }



        session.close();

        connection.close();

        context.close();

    }

}
```

Este exemplo presume que você tem configurado um servidor de mensagens JMS e uma fila chamada jms/Queue através de JNDI (Java Naming and Directory Interface).

Na próxima seção, vamos estudar o que é JNDI.

JNDI

Java Naming and Directory Interface (JNDI) é uma API Java que permite aos aplicativos descobrir e buscar objetos e serviços distribuídos em uma rede. É parte integrante da arquitetura Java Enterprise Edition (Java EE) e serve como uma ponte entre a aplicação e os recursos externos. Veja os principais objetivos e usos do JNDI:

Objetivos:

1. Descoberta de Serviços: JNDI fornece meios para descobrir serviços como bases de dados, filas de mensagens, tópicos e outros recursos empresariais.
2. Desacoplamento: Ele age como um intermediário entre o código da aplicação e os recursos, permitindo que as aplicações sejam menos dependentes de implementações específicas.
3. Flexibilidade e Portabilidade: As aplicações podem ser facilmente movidas entre diferentes ambientes e servidores de aplicação sem a necessidade de modificar o código-fonte.

Uso:

1. Naming Service (Serviço de Nomes): O JNDI fornece um serviço de nomes que permite aos objetos serem registrados e localizados através de nomes lógicos. Isso é útil para localizar recursos de forma independente de sua localização física.
2. Directory Service (Serviço de Diretório): Além do serviço de nomes, o JNDI oferece um serviço de diretório para ler e atualizar atributos associados aos nomes. Isso é frequentemente utilizado em aplicações mais complexas que exigem metainformações adicionais sobre os recursos.

Componentes Principais:

1. InitialContext: Este é o ponto de entrada para qualquer operação de pesquisa JNDI. Ele é instanciado com propriedades específicas do ambiente que fornecem detalhes sobre o serviço de nomes e diretório.
2. NamingEnumeration: É utilizado para percorrer os elementos em um contexto ou diretório.
3. DirContext e Context: São interfaces que fornecem métodos para pesquisar, criar, remover e modificar objetos no serviço de nomes ou diretório.

11225354498 - Maria Vitória Gonçalves Vieira

Exemplo

```
import javax.naming.Context;

import javax.naming.InitialContext;

import javax.sql.DataSource;



public class JNDIExample {

  public static void main(String[] args) {

    try {

      // Inicializar o contexto

      Context context = new InitialContext();



      // Buscar o recurso

      DataSource dataSource = (DataSource)
context.lookup("java:comp/env/jdbc/myDB");



      // Usar o recurso

      // ... operações de banco de dados ...



    } catch (Exception e) {

      e.printStackTrace();

    }

  }

}
```

Neste exemplo, um recurso DataSource é procurado através do JNDI. O recurso é definido em algum lugar no servidor de aplicação e exposto via JNDI para ser utilizado pelos componentes da aplicação.

Aposta estratégica

A ideia desta seção é apresentar os pontos do conteúdo que mais possuem chances de serem cobrados em prova, considerando o histórico de questões da banca em provas de nível semelhante à nossa, bem como as inovações no conteúdo, na legislação e nos entendimentos doutrinários e jurisprudenciais[1].

A Java Transaction API (JTA) é uma das tecnologias fundamentais para gerenciamento de transações em ambientes Java EE. A JTA permite que aplicações empresariais coordenem transações em múltiplos recursos de maneira distribuída, assegurando a consistência e integridade dos dados em diferentes sistemas e bancos de dados. Com JTA, as operações podem ser agrupadas em uma única unidade de trabalho, garantindo que todas as operações sejam completadas com sucesso ou que todas falhem como um todo, conhecido como atomicidade. Isso é especialmente útil em sistemas complexos onde a manipulação de vários recursos é necessária para completar uma tarefa.

Além da atomicidade, a JTA suporta os outros princípios de transações ACID: consistência, isolamento e durabilidade. A JTA garante que as mudanças em uma transação sejam permanentes e sobrevivam a falhas do sistema, o que é alcançado através de técnicas como o logging de transações e recuperação de falhas. A especificação também define padrões para o controle e a recuperação de transações, proporcionando um mecanismo robusto para resolver situações de deadlock e para lidar com situações onde os recursos podem ser bloqueados por transações concorrentes. Isso ajuda a manter a alta disponibilidade e a performance em aplicações empresariais, tornando a JTA uma parte integral da arquitetura de sistemas robustos e confiáveis.

Questões Estratégicas

Nesta seção, apresentamos e comentamos uma amostra de questões objetivas selecionadas estrategicamente: são questões com nível de dificuldade semelhante ao que você deve esperar para a sua prova e que, em conjunto, abordam os principais pontos do assunto.

[1] Vale deixar claro que nem sempre será possível realizar uma aposta estratégica para um determinado assunto, considerando que às vezes não é viável identificar os pontos mais prováveis de serem cobrados a partir de critérios objetivos ou minimamente razoáveis.

A ideia, aqui, não é que você fixe o conteúdo por meio de uma bateria extensa de questões, mas que você faça uma boa revisão global do assunto a partir de, relativamente, poucas questões.

1. (FCC/2015/Auditor de Controle Externo (TCM-GO)/Informática) Um Analista de Controle Externo da área de TI do TCM/GO está trabalhando em uma aplicação web utilizando a plataforma Java EE. Ciente que os componentes desta aplicação, para serem processados no servidor de aplicações, terão que ser implantados (deployed) em contêineres apropriados, ele esclareceu à equipe de desenvolvimento que servlets, JavaServer Pages, páginas HTML e Java Beans serão implantados e processados no contêiner A. Além disso, alguns componentes serão implantados e processados no contêiner B, destinado a prover a infraestrutura necessária para a execução de componentes de negócio distribuídos que estendem as funcionalidades de um servidor, permitindo encapsular lógica de negócio e dados específicos de uma aplicação.

Os contêineres A e B são conhecidos, respectivamente, como

a) local container e remote container.

b) web container e EJB container.

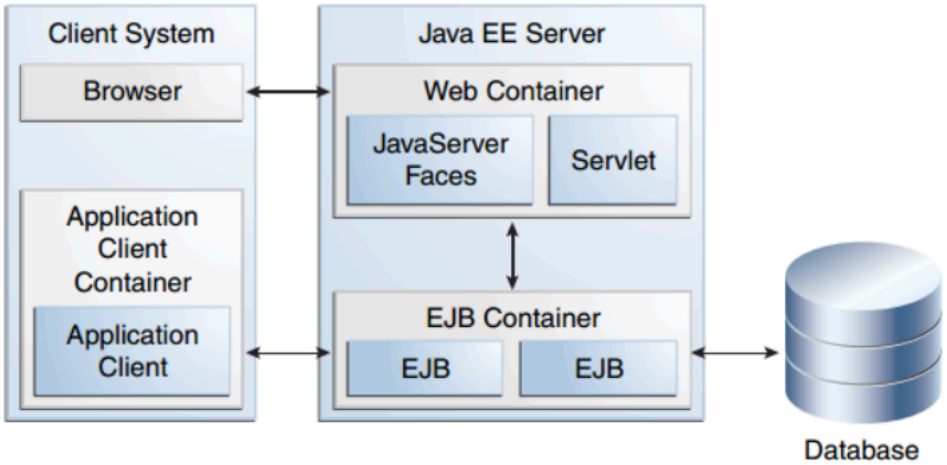
c) glassfish container e tomcat container.

d) EJB container e web container.

e) server container e client container.

Comentários

Letra B – Correta. O Container Web processam os Servlets, JSP, HTML, etc. O Container EJB processam os EJB's, que são os componentes de negócio.



Letra A – Incorreta. Os containers da plataforma JEE são Web e EJB.

11225354498 - Maria Vitória Gonçalves Vieira

Letra C – Incorreta. Glassfish é um servidor que implementa o JEE, ao passo que o Tomcat é um Container Web.

Letra D – Incorreta. A alternativa inverteu os containers. Vide resposta da letra B.

Letra E – Incorreta. Os containers da plataforma JEE são Web e EJB. Vide resposta da letra B.

2. (FCC/2019/Técnico em Web Design da Fazenda Municipal (Manaus)) Em uma Servlet de uma aplicação Java EE construída em ambiente ideal há dois métodos com estrutura praticamente idêntica, mostrados abaixo.

```
@Override
protected void ..... (HttpServletRequest request, HttpServletResponse response)
      throws ServletException, IOException {
  processRequest(request, response);
}
```

I

```
@Override
protected void ..... HttpServletRequest request, HttpServletResponse response)
      throws ServletException, IOException {
  processRequest(request, response);
}
```

II

As lacunas I e II devem ser corretamente preenchidas por

a) *doGet*, que receberá no servidor dados que chegam por meio de uma requisição HTTP GET, e *doPost*, que receberá dados que chegam por meio de uma requisição HTTP POST.

b) *servletIn*, que receberá no servidor requisições de páginas web, e *servletOut*, que retornará ao cliente dados resultantes de processamentos executados na servlet.

c) *request*, que receberá dados originados nas páginas web clientes, e *response*, que responderá a essas requisições, retornado dados após serem processados.

d) *httpFilter*, que filtrará e validará os dados de entrada originados nas páginas web, e *httpAccept*, que receberá dados que não precisam de validação.

e) *httpServlet*, que receberá dados de requisições originadas em páginas web e *interceptFilter* que verificará se os dados de entrada estão de acordo com os formatos esperados.

Comentários

Letra A – Correta. A classe HttpServlet define diversos métodos para lidar com Requisições HTTP, inclusive doGet e doPost.

11225354498 - Maria Vitória Gonçalves Vieira

Se a requisição HTTP for feita utilizando o método GET, o método doGet da servlet será invocado. O mesmo para o método doPost.

Letra B – Incorreta. Vide resposta da letra A.

Letra C – Incorreta. request (HttpServletRequest) e response (HttpServletResponse) são os parâmetros dos métodos que representam as requisições HTTP. A descrição desses parâmetros, na alternativa, está correta.

Letra D – Incorreta. Vide resposta da letra A.

Letra E – Incorreta. Vide resposta da letra A.

3. (FCC/2019/Técnico de Tecnologia da Informação da Fazenda Municipal (Manaus)) Um programador deseja criar em uma página JSP um link onde quando a palavra Excluir for clicada será feita uma chamada a uma classe servlet chamada Dados que está no servidor, passando como parâmetro um código contido na variável cod. Para isso deverá usar a instrução

a) Excluir .

b) <a href="Dados?codigo=<%!cod%>"> Excluir .

c) <a href="Dados?codigo=<%=cod%>"> Excluir .

d) Excluir .

e) <a href="Dados&codigo=<%cod%>"> Excluir .

Comentários

Letra C – Correta. Expressões são utilizadas para embutir o resultado de uma expressão na Página JSP, que é exatamente o que a questão quer fazer.

```
//Expressões JSP

<%= idade = idade + 1 %>
<%= "Esse é seu aniversário de " + idade + "anos" %>
```

Letra A – Incorreta. Vide resposta da letra C.

Letra B – Incorreta. Vide resposta da letra C.

Letra D – Incorreta. Vide resposta da letra C.

Letra E – Incorreta. Vide resposta da letra C.

4. (FCC/2019/Analista Judiciário (TRF 3ª Região)/Apoio Especializado/Informática) Em uma página JSP, para que se possa instanciar objetos da classe Cliente, que se encontra no pacote modelo, será necessário importar esta classe utilizando-se a instrução

a) <%@import src="modelo.Cliente"%>

b) <%@import package="model" class="Cliente"%>

c) <%@page import="modelo.Cliente"%>

d) <%!import("modelo.Cliente")%>

e) <%@import href="modelo.Cliente"%>

Comentários

Letra C – Correta. Diretivas são utilizadas para importar classes de um pacote, inserir dados de arquivos externos e habilitar o uso de bibliotecas de tags.

- page define atributos de configuração da página JSP.
- include inclui recursos estáticos em uma Página JSP.
- taglib estende o conjunto de tags através de uma biblioteca de tags.

A importação de classes é feita com a diretiva page:

```
//Diretiva PAGE

<%@ page import = "java.swing.*" %>

// Page pode usar 11 atributos: Info; Language; ContentType; Extends; Import; Session; Buffer;
AutoFlush; isThreadSafe; errorPage; isErrorPage.
```

Letra A – Incorreta. Vide resposta da letra C.

Letra B – Incorreta. Vide resposta da letra C.

Letra D – Incorreta. Vide resposta da letra C.

Letra E – Incorreta. Vide resposta da letra C.

5. (FCC/2018/Analista em Gestão (DPE AM)/Especializado em Tecnologia da Informação de Defensoria/Analista de Sistema) Em uma aplicação que utiliza JPA, uma classe de entidade Processo.java relaciona-se com a tabela processo em um banco de dados relacional. Assim, essa classe de entidade precisa ter algumas anotações especiais, como @Entity que especifica que se trata de uma classe de entidade, e

a) @Field, que especifica que um atributo refere-se a um campo da tabela do banco de dados.

b) @Override, que especifica que o método da classe de entidade executará uma operação SQL na tabela do banco de dados.

c) @SetSQL, que especifica que o método setter executará uma operação de inclusão ou alteração de dados na tabela do banco de dados.

d) @EntityManager, que especifica que a classe é uma entidade gerenciada por uma unidade de persistência.

11225354498 - Maria Vitória Gonçalves Vieira

e) @Id, que indica qual atributo na classe de entidade representa a chave primária na tabela do banco de dados.

Comentários

Letra E – Correta. A anotação @Id indica que o atributo é chave primária da entidade.

Letra A – Incorreta. A anotação @Field não existe na JPA.

Letra B – Incorreta. A anotação @Override não existe na JPA.

Letra C – Incorreta. A anotação @SetSQL não existe na JPA.

Letra D – Incorreta. A anotação @EntityManager não existe na JPA.

6. (FCC/2016/Assistente Tecnológico (PRODATER)/Programador) Em uma aplicação que utiliza JPA e Hibernate, uma classe de entidade do banco de dados possui anotações para mapear atributos da classe para campos de uma tabela. Estas anotações são inseridas na linha anterior à declaração do atributo. Considerando que o atributo id da classe tenha sido declarado pela instrução private Long id; para indicar que este atributo representa a chave primária da tabela, e que é auto incrementável, utilizam-se, respectivamente, as anotações

a) @PrimaryKey e @GeneratedValue

b) @Primary e @Append

c) @Id e @GeneratedValue

d) @PrimaryKey e @AutoIncrement

e) @Id e @AutoIncrement

Comentários

Letra C – Correta. A anotação @Id especifica o identificador da entidade. As entidades devem ter um atributo identificador. Já a anotação @GeneratedValue indica que o valor do identificador da entidade é gerado automaticamente utilizando, por exemplo, uma sequence do banco de dados.

Letra A – Incorreta. A anotação @PrimaryKey não faz parte do JPA/Hibernate. A anotação @GeneratedValue realmente indica que o valor do identificador da entidade é gerado automaticamente utilizando, por exemplo, uma sequence do banco de dados.

Letra B – Incorreta. As anotações @Primary e @Append não fazem parte do JPA/Hibernate.

Letra D – Incorreta. As anotações @PrimaryKey e @AutoIncrement não fazem parte do JPA/Hibernate.

11225354498 - Maria Vitória Gonçalves Vieira

Letra E – Incorreta. A anotação @Id realmente especifica o identificador da entidade. Já a anotação @AutoIncrement não faz parte do JPA/Hibernate.

7. (FCC/2015/Técnico Judiciário (TRE PB)/Apoio Especializado/Programação de Sistemas) Em aplicações que usam JPA, quando um objeto de uma classe de entidade do banco de dados é instanciado pela primeira vez ele está no estado novo e para passá-lo para o estado gerenciado, utiliza-se um método da interface EntityManager chamado

a) datashed.

b) managed.

c) persist.

d) flush.

e) commit.

Comentários

Letra C – Correta. O método **persist()** da interface EntityManager torna uma instância gerenciada e persistente.

Letra A – Incorreta. A interface EntityManager não possui o método **datashed()**.

Letra B – Incorreta. A interface EntityManager não possui o método **managed()**.

Letra D – Incorreta. Os métodos persist(), merge() e remove() aguardam finalização da transação para sincronizar as entidades com o banco de dados, o método **flush()** força essa sincronização no momento em que é chamado.

Letra E – Incorreta. A interface EntityManager não possui o método **commit()**.

8. (FCC/2015/Técnico Judiciário (TRT 9ª Região)/Apoio Especializado/Tecnologia da Informação) Em uma aplicação que utiliza JPA e Hibernate, no arquivo persistence.xml

a) são definidas as configurações do servidor de aplicação, como número de conexões simultâneas permitidas e dados de log.

b) é definido o mapeamento objeto-relacional entre as tabelas do banco de dados e as classes de entidade da aplicação.

c) são definidas as queries nomeadas, que são queries pré-definidas que podem ser chamadas pelo nome a partir de classes de acesso a dados da aplicação.

d) há o mapeamento de componentes da camada de apresentação para os respectivos componentes da camada de acesso a dados da aplicação.

e) são definidas as propriedades de conexão com o banco de dados, como o driver JDBC, a URL de conexão, o nome do usuário e a senha.

11225354498 - Maria Vitória Gonçalves Vieira

Comentários

Letra E – Correta. O arquivo persistence.xml é um arquivo de configuração cujo conteúdo tem informações como a url de conexão com o banco de dados, usuário, senha, etc.

Letra A – Incorreta. Vide resposta da letra E.

Letra B – Incorreta. O mapeamento é feito utilizando anotações. No caso do Hibernate, também pode ser feito utilizando arquivos hbm.xml.

Letra C – Incorreta. Vide resposta da letra E.

Letra D – Incorreta. Vide resposta da letra E.

9. (FCC/2015/Técnico Judiciário (TRT 15ª Região)/Apoio Especializado/Tecnologia da Informação) Em uma aplicação web que utiliza Enterprise JavaBeans – EJB para implementar um carrinho de compras, utilizou-se um tipo de bean que mantém o estado durante uma sessão com o cliente. Nesta aplicação, para indicar ao servidor que a classe é um bean com estado de sessão deve-se utilizar, antes da declaração da classe, a anotação

a) @Session state="true"

b) @Stateful

c) @SessionState= "true"

d) @Stateless

e) @SessionRemote

Comentários

Letra B – Correta. A anotação @Stateful especifica um Stateful Session Bean, que representa objetos de negócio que mantém estado entre chamadas de métodos.

Letra A – Incorreta. A especificação EJB não possui a anotação @Session. A anotação correta é @Stateful.

Letra C – Incorreta. A especificação EJB não possui a anotação @SessionState. A anotação correta é @Stateful.

Letra D – Incorreta. A anotação @Stateless especifica um Stateless Session Bean, que representa objetos de negócio que não mantém estado entre invocações de métodos.

Letra E – Incorreta. A especificação EJB não possui a anotação @SessionRemote. A anotação correta é @Stateful. A anotação @Remote especifica uma interface remota de um EJB.

11225354498 - Maria Vitória Gonçalves Vieira

QUESTIONÁRIO DE REVISÃO E APERFEIÇOAMENTO

A ideia do questionário é elevar o nível da sua compreensão no assunto e, ao mesmo tempo, proporcionar uma outra forma de revisão de pontos importantes do conteúdo, a partir de perguntas que exigem respostas subjetivas.

São questões um pouco mais desafiadoras, porque a redação de seu enunciado não ajuda na sua resolução, como ocorre nas clássicas questões objetivas.

O objetivo é que você realize uma auto explicação mental de alguns pontos do conteúdo, para consolidar melhor o que aprendeu ;)

Além disso, as questões objetivas, em regra, abordam pontos isolados de um dado assunto. Assim, ao resolver várias questões objetivas, o candidato acaba memorizando pontos isolados do conteúdo, mas muitas vezes acaba não entendendo como esses pontos se conectam.

Assim, no questionário, buscaremos trazer também situações que ajudem você a conectar melhor os diversos pontos do conteúdo, na medida do possível.

É importante frisar que não estamos adentrando em um nível de profundidade maior que o exigido na sua prova, mas apenas permitindo que você compreenda melhor o assunto de modo a facilitar a resolução de questões objetivas típicas de concursos, ok?

Nosso compromisso é proporcionar a você uma revisão de alto nível!

Vamos ao nosso questionário:

Perguntas

1. O que diferencia Java EE de Jakarta EE?
2. Quais são as principais responsabilidades do Entity Manager na JPA?
3. O que é Two-Phase Commit em JTA?
4. Quais são os estados do ciclo de vida de uma entidade JPA?
5. Como o Container EJB gerencia o ciclo de vida de um EJB?
6. O que é Inversão de Controle (IoC) no contexto de EJB?
7. Qual a principal vantagem de usar mensagens assíncronas em JMS?
8. Quais são os modelos de comunicação suportados pelo JMS?

9. O que é EFSP e qual é sua importância?
10. Como JavaServer Pages (JSP) e Servlets se relacionam?
11. O que é JPQL e em que contexto ele é usado?
12. Como a Criteria API se diferencia do JPQL?
13. Quais são os tipos de Enterprise JavaBeans (EJB)?
14. Qual é a função das anotações no contexto da JPA?
15. O que é JNDI e para que é utilizado?
16. Como o Container EJB lida com a segurança?
17. Quais são as responsabilidades do Servlet Container?
18. Como o JTA gerencia transações?
19. O que é Scriplet em JSP?
20. O que são Beans Gerenciados (Managed Beans) no contexto de JSF?

Perguntas e Respostas

1. O que diferencia Java EE de Jakarta EE?
   Resposta: Jakarta EE é a evolução do Java EE e está sob o comando da Eclipse Foundation. Visa modernizar e desenvolver as tecnologias empresariais baseadas em Java.
2. Quais são as principais responsabilidades do Entity Manager na JPA?
   Resposta: O Entity Manager é responsável por gerenciar o ciclo de vida das entidades, incluindo persistência, recuperação, atualização e exclusão de registros no banco de dados.
3. O que é Two-Phase Commit em JTA?
   Resposta: Two-Phase Commit é um protocolo que garante que uma transação distribuída seja processada atomicamente, envolvendo duas fases: a fase de preparação e a fase de commit ou rollback.
4. Quais são os estados do ciclo de vida de uma entidade JPA?
   Resposta: Os estados são Novo, Gerenciado, Removido e Destacado.
5. Como o Container EJB gerencia o ciclo de vida de um EJB?
   Resposta: O Container EJB gerencia automaticamente a criação, ativação, passivação e destruição de EJBs.
6. O que é Inversão de Controle (IoC) no contexto de EJB?
   Resposta: IoC é um padrão de design em que o controle do fluxo da aplicação é delegado ao container EJB, permitindo uma melhor separação de preocupações.
7. Qual a principal vantagem de usar mensagens assíncronas em JMS?
   Resposta: A principal vantagem é a desacoplagem entre o produtor e o consumidor, permitindo maior escalabilidade e robustez.

11225354498 - Maria Vitória Gonçalves Vieira

8. Quais são os modelos de comunicação suportados pelo JMS?
   Resposta: Point-to-Point (P2P) e Publish-Subscribe (Pub-Sub).
9. O que é EFSP e qual é sua importância?
   Resposta: EFSP é o Eclipse Foundation Specification Process, um processo utilizado para o desenvolvimento de especificações, incluindo Jakarta EE. É importante para manter a qualidade e a padronização.
10. Como JavaServer Pages (JSP) e Servlets se relacionam?
    Resposta: JSPs são compilados em Servlets. Ambos são usados para desenvolver aplicações web, mas JSP é mais focado na apresentação, enquanto Servlets lidam com a lógica do lado do servidor.
11. O que é JPQL e em que contexto ele é usado?
    Resposta: JPQL é a Java Persistence Query Language, usada para fazer queries em entidades gerenciadas pela JPA.
12. Como a Criteria API se diferencia do JPQL?
    Resposta: Criteria API permite a criação de queries dinâmicas, enquanto JPQL é geralmente usado para queries estáticas.
13. Quais são os tipos de Enterprise JavaBeans (EJB)?
    Resposta: Os tipos principais são Session Beans, Message-Driven Beans e Entity Beans (embora estes últimos sejam considerados obsoletos em favor da JPA).
14. Qual é a função das anotações no contexto da JPA?
    Resposta: As anotações são usadas para configurar o mapeamento objeto-relacional e outras configurações da JPA.
15. O que é JNDI e para que é utilizado?
    Resposta: JNDI é a Java Naming and Directory Interface, uma API para descoberta e busca de serviços e objetos em uma rede.
16. Como o Container EJB lida com a segurança?
    Resposta: O Container EJB fornece suporte a autenticação e controle de acesso orientado a papéis.
17. Quais são as responsabilidades do Servlet Container?
    Resposta: O Servlet Container é responsável pelo ciclo de vida dos Servlets, roteamento de requisições, e também pode fornecer recursos como pooling de threads e segurança.
18. Como o JTA gerencia transações?
    Resposta: O JTA coordena e gerencia transações distribuídas, lidando com o início, commit e rollback de transações envolvendo múltiplos recursos.
19. O que é Scriplet em JSP?
    Resposta: Um Scriplet é um trecho de código Java incrustado em um documento JSP, geralmente usado para programação procedural dentro da página.
20. O que são Beans Gerenciados (Managed Beans) no contexto de JSF?
    Resposta: Beans Gerenciados são beans que são gerenciados pelo container JSF e



11225354498 - Maria Vitória Gonçalves Vieira

podem ser acessados a partir de páginas JSF. Eles atuam como um modelo para a UI e podem conter lógica de negócios.

Lista de Questões Estratégicas

1. (FCC/2015/Auditor de Controle Externo (TCM-GO)/Informática) Um Analista de Controle Externo da área de TI do TCM/GO está trabalhando em uma aplicação web utilizando a plataforma Java EE. Ciente que os componentes desta aplicação, para serem processados no servidor de aplicações, terão que ser implantados (deployed) em contêineres apropriados, ele esclareceu à equipe de desenvolvimento que servlets, JavaServer Pages, páginas HTML e Java Beans serão implantados e processados no contêiner A. Além disso, alguns componentes serão implantados e processados no contêiner B, destinado a prover a infraestrutura necessária para a execução de componentes de negócio distribuídos que estendem as funcionalidades de um servidor, permitindo encapsular lógica de negócio e dados específicos de uma aplicação.

Os contêineres A e B são conhecidos, respectivamente, como

a) local container e remote container.

b) web container e EJB container.

c) glassfish container e tomcat container.

d) EJB container e web container.

e) server container e client container.

2. (FCC/2019/Técnico em Web Design da Fazenda Municipal (Manaus)) Em uma Servlet de uma aplicação Java EE construída em ambiente ideal há dois métodos com estrutura praticamente idêntica, mostrados abaixo.

11225354498 - Maria Vitória Gonçalves Vieira

```
@Override
protected void ..... (HttpServletRequest request, HttpServletResponse response)
      throws ServletException, IOException {
  processRequest(request, response);
}

@Override
protected void ..... HttpServletRequest request, HttpServletResponse response)
      throws ServletException, IOException {
  processRequest(request, response);
}
```

I

II

As lacunas I e II devem ser corretamente preenchidas por

a) *doGet*, que receberá no servidor dados que chegam por meio de uma requisição HTTP GET, e *doPost*, que receberá dados que chegam por meio de uma requisição HTTP POST.

b) *servletIn*, que receberá no servidor requisições de páginas web, e *servletOut*, que retornará ao cliente dados resultantes de processamentos executados na servlet.

c) *request*, que receberá dados originados nas páginas web clientes, e *response*, que responderá a essas requisições, retornado dados após serem processados.

d) *httpFilter*, que filtrará e validará os dados de entrada originados nas páginas web, e *httpAccept*, que receberá dados que não precisam de validação.

e) *httpServlet*, que receberá dados de requisições originadas em páginas web e *interceptFilter* que verificará se os dados de entrada estão de acordo com os formatos esperados.

3. (FCC/2019/Técnico de Tecnologia da Informação da Fazenda Municipal (Manaus)) Um programador deseja criar em uma página JSP um link onde quando a palavra Excluir for clicada será feita uma chamada a uma classe servlet chamada Dados que está no servidor, passando como parâmetro um código contido na variável cod. Para isso deverá usar a instrução

a) Excluir .

b) <a href="Dados?codigo=<%!cod%>"> Excluir .

c) <a href="Dados?codigo=<%=cod%>"> Excluir .

d) Excluir .

e) <a href="Dados&codigo=<%cod%>"> Excluir .

4. (FCC/2019/Analista Judiciário (TRF 3ª Região)/Apoio Especializado/Informática) Em uma página JSP, para que se possa instanciar objetos da classe Cliente, que se

11225354498 - Maria Vitória Gonçalves Vieira

encontra no pacote modelo, será necessário importar esta classe utilizando-se a instrução

a) <%@import src="modelo.Cliente"%>

b) <%@import package="model" class="Cliente"%>

c) <%@page import="modelo.Cliente"%>

d) <%!import("modelo.Cliente")%>

e) <%@import href="modelo.Cliente"%>

5. (FCC/2018/Analista em Gestão (DPE AM)/Especializado em Tecnologia da Informação de Defensoria/Analista de Sistema) Em uma aplicação que utiliza JPA, uma classe de entidade Processo.java relaciona-se com a tabela processo em um banco de dados relacional. Assim, essa classe de entidade precisa ter algumas anotações especiais, como @Entity que especifica que se trata de uma classe de entidade, e

a) @Field, que especifica que um atributo refere-se a um campo da tabela do banco de dados.

b) @Override, que especifica que o método da classe de entidade executará uma operação SQL na tabela do banco de dados.

c) @SetSQL, que especifica que o método setter executará uma operação de inclusão ou alteração de dados na tabela do banco de dados.

d) @EntityManager, que especifica que a classe é uma entidade gerenciada por uma unidade de persistência.

e) @Id, que indica qual atributo na classe de entidade representa a chave primária na tabela do banco de dados.

6. (FCC/2016/Assistente Tecnológico (PRODATER)/Programador) Em uma aplicação que utiliza JPA e Hibernate, uma classe de entidade do banco de dados possui anotações para mapear atributos da classe para campos de uma tabela. Estas anotações são inseridas na linha anterior à declaração do atributo. Considerando que o atributo id da classe tenha sido declarado pela instrução private Long id; para indicar que este atributo representa a chave primária da tabela, e que é auto incrementável, utilizam-se, respectivamente, as anotações

a) @PrimaryKey e @GeneratedValue

b) @Primary e @Append

c) @Id e @GeneratedValue

d) @PrimaryKey e @AutoIncrement

e) @Id e @AutoIncrement



11225354498 - Maria Vitória Gonçalves Vieira

7. (FCC/2015/Técnico Judiciário (TRE PB)/Apoio Especializado/Programação de Sistemas) Em aplicações que usam JPA, quando um objeto de uma classe de entidade do banco de dados é instanciado pela primeira vez ele está no estado novo e para passá-lo para o estado gerenciado, utiliza-se um método da interface EntityManager chamado

a) datashed.

b) managed.

c) persist.

d) flush.

e) commit.

8. (FCC/2015/Técnico Judiciário (TRT 9ª Região)/Apoio Especializado/Tecnologia da Informação) Em uma aplicação que utiliza JPA e Hibernate, no arquivo persistence.xml

a) são definidas as configurações do servidor de aplicação, como número de conexões simultâneas permitidas e dados de log.

b) é definido o mapeamento objeto-relacional entre as tabelas do banco de dados e as classes de entidade da aplicação.

c) são definidas as queries nomeadas, que são queries pré-definidas que podem ser chamadas pelo nome a partir de classes de acesso a dados da aplicação.

d) há o mapeamento de componentes da camada de apresentação para os respectivos componentes da camada de acesso a dados da aplicação.

e) são definidas as propriedades de conexão com o banco de dados, como o driver JDBC, a URL de conexão, o nome do usuário e a senha.

9. (FCC/2015/Técnico Judiciário (TRT 15ª Região)/Apoio Especializado/Tecnologia da Informação) Em uma aplicação web que utiliza Enterprise JavaBeans – EJB para implementar um carrinho de compras, utilizou-se um tipo de bean que mantém o estado durante uma sessão com o cliente. Nesta aplicação, para indicar ao servidor que a classe é um bean com estado de sessão deve-se utilizar, antes da declaração da classe, a anotação

a) @Session state="true"

b) @Stateful

c) @SessionState= "true"

d) @Stateless

e) @SessionRemote

11225354498 - Maria Vitória Gonçalves Vieira

Gabaritos

1. B
2. A
3. C
4. C
5. E
6. C
7. C
8. E
9. B

ESSA LEI TODO MUNDO CONHECE:
PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.

1. 

Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.

2. 

Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).

3. 

Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).

4. 

Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).

5. 

Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).

6. 

Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).

7. 

Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.

8. 

O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.